

Autonomous Data Quality Monitoring with AI Agents: Integrating ML with Cloud Warehouses and Data Lakes

¹ Srinivas Chippagiri
Sr. Member of Technical Staff
Seattle, WA, USA 98103
0009-0004-9456-3951
cvas22@gmail.com

² Karan Alang
Independent Researcher
Cupertino, CA, USA 95014
0009-0001-3284-3155
karan.alang@gmail.com

³ Ankush Gumber
Independent Researcher
Livermore, CA, USA 94550
0009-0009-7733-9137
er.ankushgumber@gmail.com

⁴ Sooraj George Thomas
Independent Researcher
Austin, Texas, USA, 78641
0009-0004-0729-1151
soorajgthomas@gmail.com

Abstract—since storing data in cloud warehouses and lakes has increased, it is now important to rely on automation for keeping data accurate and trustworthy. It outlines an autonomous way of monitoring data quality by using AI agents with botnet techniques and cloud architecture. Key features of the system are that it constantly scans the data, finds unusual activities, detects any changes in the structure, and responds to any data updates without someone needing to act. Missed values, duplication, inconsistency, and noticing outliers on both structured and semi-structured data is done by using both supervised and unsupervised ML models. AI agents interact by way of a network pipeline and use reinforcement learning to pick the best solutions. easily set up Cloud Genomics in any leading cloud environment, and it works well with data warehouses as well as data lakes. Experimental evidence shows that AI-driven analysis is better at detecting errors, offering faster results, and being more adaptable than hand-made and rule-based ones. In this research, data pipelines can become more flexible and easier to manage in today's business architecture.

Keywords— *Data quality, AI agents, machine learning, data lakes, cloud data warehouse, autonomous systems, data monitoring, anomaly detection*

I. INTRODUCTION

Now that so much data is being used, cloud data warehouses and data lakes play an important role by letting businesses handle a lot of data without much effort or expense. Because organizations depend so much on data systems for important decisions, the quality of the data has to be top priority. If data is poor, the company can expect to get misleading insights, unreliable models, and wrong results from analytics [1], [2].

OSQL systems, based on rules alone, are usually not scalable, flexible, or up to handling the growing amounts, forms, and reliability of current data [3]. Such systems require a lot of hands-on work and are not meant to detect when the structure of data starts to diverge, data arrives after a deadline, or there are semantic issues [4]. Due to machine learning (ML) and AI, it is now possible to develop systems that can detect, fix, and keep an eye on DQ anomalies without being controlled [5].

Lately, AI agent-based systems have demonstrated the capability to take care of complicated workflows such as identifying anomalies, tracking the causes of problems, and self-repair [6], [7]. If these agents are integrated into cloud infrastructure, they monitor and identify issues in structured and semi-structured data from all kinds of sources, and solve them with statistical and machine learning algorithms on their own [8].

Also, using event-driven architectures and RL, these agents are able to adapt to new elements in the data and changing business regulations [9]. RL-based agents pick up the best solutions for fixing issues through observation and practice, which results in a very useful and adaptable quality assurance setup.

The paper suggests an automated system that depends on AI agents to manage data quality in various cloud data warehouses and lakes. We have made contributions in the following areas.

A way to use intelligent agents that can be easily deployed into data pipelines.

- By using machine learning, it is possible to run profile checks, detect suspicious events, and ensure the schemas are correct.
- Use of reinforcement learning to support adaptive ways of fixing issues.
- Working with large-scale datasets to show that the system has better results, faster operations, and a lower cost of running.

Bringing together data engineering and AI automation in our solution, we ensure businesses have trustworthy, strong, and self-reliant data systems that adjust to more complex data use.

II. RELATED WORKS

Long ago, data warehousing and analytics experts recognized that high DQ is very important. At first, cleaning data relied on manually setting rules and using only deterministic tools, so it required specialized knowledge and was not able to adapt to high-speed changes [1], [3]. They mostly looked for errors in the syntax, holes where details are missing, and simple duplicates, but could not catch semantic errors, growing drifts, or those that depend on the context.

These days, more focus is given to ML-driven ways to ensure data quality, using unsupervised learning for detecting any suspicious data patterns and clustering methods to spot any data outside of the groups [5], [10]. In this regard, DeepDQ set out by Rekatsinas et al. combines graphical models and machine learning analysis to enhance the system's detection accuracy [5]. In the same way, the DeepClean and ActiveClean frameworks allow data cleaning using user feedback and predictions.

The introduction of data lakes into businesses brought extra issues to handling DQ because the data there isn't always nicely organized. Gao et al. pointed out three issues in working with data lakes: metadata, upgrading versions, and inferring schemas; to address these concerns, new research began on adaptive profiling and semantic tagging [12].

Intelligent agents in autonomous systems are becoming more popular in DQ. Kitchenham et al. [6] were the first to suggest agent-based monitoring, and now modern tools have adopted their idea in event-driven ways on cloud platforms. They depend on the cooperation of several agents, message delivery through publish-subscribe, and intelligent enforcement of rules to ensure good scalability in monitoring [13].

RL is considered a promising answer to the problems caused by remediation. Using RL, agents can create dynamic solutions to cope with frequent problems arising from data changing and producing shifting feedback often. Some work like AutoClean uses RL to choose the best way to clean different surfaces as time goes by [14]. Some people concentrating on detecting disturbances [10] while others focus on automated decisions in uncertain cases.

Still, most existing technologies do not fully connect to cloud data warehouses and data lakes, making the process of monitoring fragmented. By using our framework, an AI-agent learns to find and resolve issues with data quality all by itself, always adapting to new kinds of data and changes in structure.

III. SYSTEM ARCHITECTURE

The designed system is meant to work automatically on many types of data platforms in the cloud, such as Snowflake, BigQuery, Amazon S3, or Delta Lake, to monitor and enhance data quality. By using edge agents and event-driven methods, the architecture makes sure that AI-enabled data quality processes take place effectively at any scale, as shown in Fig 1.

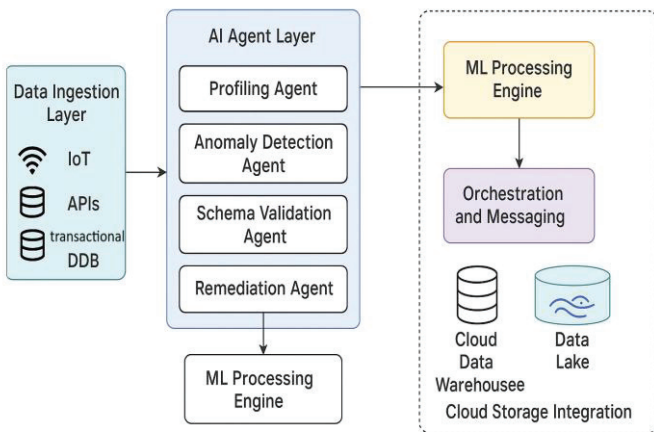


Fig. 1. Autonomous Data Quality Monitoring with AI Agents

A. Overview

At a high level, the architecture consists of five primary components:

- **Data Ingestion Layer** The main function is to pick up data as batches or in a continuous flow from various sources and store it in the cloud.
- **AI Agent Layer** An agent system in which every agent is assigned to specific duties such as profiling data, identifying other data issues, checking the schema, or repairing the data.
- **ML Processing Engine** They gave consolidated reports about outliers, anomalies, and changes in the data by using supervised and unsupervised models. They are taught using old records of data quality.
- **Cloud Storage Integration** Using metadata APIs, SQL connectors, and object store APIs, it helps work with data from warehouses and lakes.
- **Orchestration and Messaging Layer** Organize and direct workflows and messages between agents by depending on event-driven platforms such as Apache Kafka, Airflow, or AWS Step Functions.

B. AI Agent Workflow

Each AI agent operates autonomously but follows a common lifecycle:

- **Trigger Detection** – Agents are event-driven (e.g., new data arrival, schema change).
- **Profiling & Feature Extraction** – Agents generate statistical summaries and metadata.
- **Anomaly Detection & Alerting** – Agents apply ML models to identify anomalies and emit alerts.
- **Remediation Planning** – Agents consult learned policies or trigger RL-based decision-making for correction.
- **Feedback Loop** – Agents record outcomes to retrain models and refine strategies.

C. Scalability and Cloud-Native Deployment

The system is set up to use containers to ease the process of adding more computing power. It is cloud-agnostic and can be deployed on AWS, GCP, or Azure, with native integration with cloud services like:

- AWS Glue, S3, Redshift
- GCP Dataflow, BigQuery, Cloud Storage
- Azure Synapse, Data Lake, ML Studio

Agents are loosely coupled and stateless, ensuring fault tolerance and high availability.

D. Security and Governance Integration

The system includes optional integration with:

- Data governance platforms (e.g., Collibra, Alation)
- IAM systems for access control
- Audit logs for compliance and traceability

Data considered sensitive is protected using encryption practices based on people's roles and secure ways of transferring data (like TLS or IAM data access).

IV. METHODOLOGY

The approach uses intelligent agents within modules to oversee the detection and correction of data quality matters on their own. Every sub-process is optimized for growth,

working well in the cloud, and adapting to changes in managing data from both structured and semi-structured data is shown in the Fig 2.

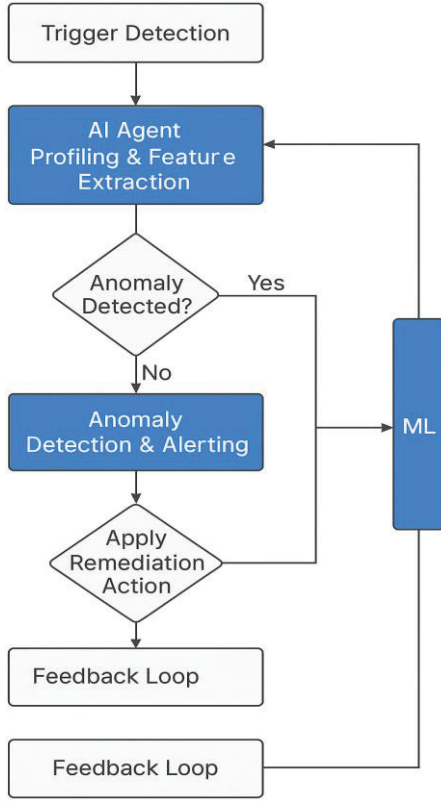


Fig. 2. flowchart of the agent workflow

The detailed explanation of the methodology is shown below:

A. Data Profiling and Feature Extraction

At the beginning, the profiling agent analyzes data in terms of its structure and numbers. Using null value frequency, mean, standard deviation, uniqueness ratio, and type conformity helps in computation. These statistics turn into feature vectors that the anomaly detection models further process.

Let $X = \{x_1, x_2, \dots, x_n\}$ represent a dataset column. Basic profiling generates:

$$\mu = \frac{1}{n} \sum_{i=1}^n x_i, \quad \sigma = \sqrt{\frac{1}{n} \sum_{i=1}^n (x_i - \mu)^2}, \quad \text{NullRate} = \frac{\# \text{nulls}}{n}$$

These features help establish dynamic data baselines.

These equations are the main metrics the profiling agent uses when making statistical profiling. The mean (μ) gives the center of a numeric column as its main value. The second is called the standard deviation (σ), as it tells how distributed the values are from the mean and also helps spot outliers or unusual differences. The third measure is the null rate, indicating the amount of missing entries in one column of the data. They are required to properly conduct anomaly detection and validate the schema during subsequent tasks.

B. ML-Based Anomaly Detection

Anomaly detection makes use of supervised (such as Decision Trees and SVM) and unsupervised (including Isolation Forest and Autoencoders) models by studying past

data. Anomalies are indicated when measurements are far from what is anticipated.

For example, Isolation Forest identifies anomalies based on average path lengths:

$$\text{AnomalyScore}(x) = 2 \frac{E(h(x))}{c(n)}$$

Where $E(h(x))$ is the average tree depth to isolate point x , and $c(n)$ is a normalization factor. Higher scores indicate more anomalous records.

Isolation Forest is an unsupervised algorithm in which this equation is used for anomaly detection. The equation says that $E(h(x))$ is the expected number of splits needed to find a data point x in a random tree, and $c(n)$ makes the results suitable for tiny sample sizes. Any data point that is far from the norm is spotted faster and is assigned a higher score. Accordingly, the system can spot unusual data, which may need to be looked at more closely or fixed.

C. Schema Drift and Validation

The agent using the schema validation platform compares the latest schema signatures to past versions registered in the schema registry. Schema drifts are detected by computing structural dissimilarity:

$$D_{\text{schema}} = \frac{|A_{\text{new}} \Delta A_{\text{ref}}|}{|A_{\text{ref}}|}$$

Where A represents attribute sets. This allows for early detection of breaking schema changes.

This means, symmetric difference between the new attributes A_{new} and the reference schema A_{ref} is used to quantify schema drift. The numerator shows the number of unique attributes, and the denominator scales the score to fit with the original dataset. A high drift score shows that something major has changed in the schema, and this could mean the agent will need to be altered or validation and alert routines might be set off

Algorithm: Schema Drift Detection

Input: New schema A_{new} , Reference schema A_{ref}
Output: Boolean value for drift detected

```

1: Compute drift score:
2:   $D = |A_{\text{new}} \Delta A_{\text{ref}}| / |A_{\text{ref}}|$  // symmetric difference
3: if  $D > \text{Threshold } T$  then
4:   return True // Drift Detected
5: else
6:   return False // No Drift
  
```

It detects changes in the data schema by checking the new data against the original one kept as a reference. The drift score is calculated according to how different the attributes are and then normalized to match the size of the reference. Only when the drift score goes past the designated threshold will the algorithm report a drift event. The approach enables speedy spotting of issues in data, for example when a field is added, taken away, or renamed, causing problems for following processes. If this stage happens early on, AI agents are more able to respond or sound alarms, ensuring the reliability of data and models based on same formats.

D. Reinforcement Learning-Based Remediation

The remediation agent uses a reinforcement learning policy $\pi(s) \rightarrow a$, where:

- s is the current data quality state
- a is the action (e.g., imputation, removal, escalate)

- Rewards are based on post-action anomaly reduction and downstream model performance
- A simplified Q-learning approach updates the agent as:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

This enables adaptive, context-aware remediation over time.

This equation is the core of Q-learning, used by the remediation agent to learn optimal data quality correction policies. Here:

- $Q(s, a)$: current value for taking action a in state s
- r : immediate reward received after the action
- γ : discount factor for future rewards
- α : learning rate

The Q-value is updated by comparing the current estimate with a new sample based on the reward and the best expected future value $\max_{a'} Q(s', a')$. This allows the agent to improve its decision-making policy over time through trial and error.

Algorithm: Q-Learning-Based Remediation Strategy

Algorithm: RL-Based Remediation Strategy for Data Quality Issues

Input: DataQualityState s , Q-table Q , Learning rate α , Discount γ , Episodes E

Output: Optimal remediation actions for future issues

```

1: Initialize  $Q(s, a)$  arbitrarily for all  $s \in S, a \in A$ 
2: for episode = 1 to  $E$  do
3:   Observe initial state  $s$ 
4:   repeat
5:     Select action  $a$  using  $\epsilon$ -greedy policy from  $Q(s, a)$ 
6:     Execute action  $a$  (e.g., impute, remove, escalate)
7:     Receive reward  $r$  (based on improvement in DQ metric)
8:     Observe new state  $s'$ 
9:     Update Q-value:
10:     $Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$ 
11:     $s \leftarrow s'$ 
12:   until terminal state or max steps

```

Q-learning, a RL method, is used by this algorithm to pick the most suitable way to rectify data quality issues. An episode in the algorithm takes as input a data pipeline state, showing the current issues in the data (for example, smartphones vs. computers reporting data in the survey). The agent decides on an action, along with an ϵ -greedy approach, which manages exploring new possibilities as well as making use of existing knowledge. When data becomes better or downstream tasks are finished without issues, a reward signal is created. Q-values are updated at every iteration to find out which is the best action for each state. With time, AI uses the latest data patterns to come up with effective ways to fix data concerns without the need for users to write additional rules.

E. Event-Driven Orchestration

Events trigger different actions in the agents. If any data processing event occurs, the right agents are triggered through message queues. Due to this, tasks can run separately and without connection unless specified, with help from Apache Kafka, AWS EventBridge, and GCP Pub/Sub.

V. IMPLEMENTATION

Because the framework is cloud-native, scalable, and made up of modules, it fits well with the technology used in most businesses. It depends on open-source software, cloud servers, and container microservices to set up AIs and control different stages of the process.

A. Technology Stack

The following technologies and frameworks were used in implementation:

- Programming Languages: Python (for ML models and agents), SQL (for profiling and metadata queries).
- ML Libraries: Scikit-learn, XGBoost, Keras (for anomaly detection and prediction models).
- Workflow Orchestration: Apache Airflow, Prefect
- Message Queues: Apache Kafka for asynchronous communication between agents.
- Cloud Platforms: AWS and GCP
 - AWS: S3, Lambda, Glue, Redshift
 - GCP: Cloud Storage, BigQuery, Dataflow
- Containerization: Docker, Kubernetes for deployment of AI agents.

B. Agent Deployment

All AI agents are made as small microservices that reside in Docker containers. Kubernetes is used to handle these containers so that scaling and error mitigation are enabled. When an event is triggered, the corresponding agent from Kafka is sent and then executes the necessary functions (such as profiling and anomaly detection).

They put intermediate values, for example feature vectors and anomaly scores, in the central metadata repository using PostgreSQL or Google Cloud Firestore so that other agents can use them and retain the context.

C. Data Integration

The system integrates with cloud data warehouses and data lakes through standardized connectors:

- Snowflake and BigQuery via JDBC connectors and SQL-based APIs
- Amazon S3, Google Cloud Storage via object store clients for direct schema extraction and profiling
- Metadata Management: Integration with Apache Atlas for lineage and schema registry for version control

Data ingestion is either batch-based (via scheduled ETL jobs) or real-time using streaming platforms like Apache Kafka or AWS Kinesis.

D. Model Training and Inference

Before deployment, the model is built with training data from the past that are known to have issues. By using k-fold cross-validation, Isolation Forest, Autoencoders, and LSTM networks are both trained and validated. After being trained, the model is stored as a file (e.g., with joblib or ONNX) to be picked up by inference agents.

As soon as additional information appears, inference is applied on it instantly. They identify the needed features, bring the model on board, and give anomaly ratings or detect any schema variations straight away. These outcomes are sent to a monitoring screen and start the appropriate response actions whenever needed.

E. Monitoring and Logging

A custom dashboard was built using Grafana and Prometheus, offering real-time visibility into:

- Data quality metrics (null rates, drift scores, anomaly counts)
- Agent health and latency
- Model confidence scores and alerts

Audit logs of remediation actions and anomaly detections are stored in a versioned log repository, supporting traceability and compliance.

VI. EXPERIMENTAL EVALUATION

Extensive tests were performed using several datasets, both real and synthetic, in cloud environments to see how well the framework works. The research team checked the accuracy of detection, the frequency of false positives, the time the system required to work, and how it adjusted to updated datasets.

A. Dataset Description

We used two types of datasets:

- Synthetic Datasets: I used libraries named PySynthetic and Faker to produce a lot of tabular data that contains controlled anomalies such as missing items, wrong types, skew, and similar records.
- Real-World Datasets:
 - *Healthcare Dataset*: MIMIC-III Hospital Admission data that is both inconsistent with its structure and meaning.
 - *E-commerce Dataset*: Sales and product information from Amazon S3, which may contain schema drift and null values, has been loaded into the data store.

There were datasets from 10K to 10M records with some CSV, JSON, and Parquet types, which were organized differently.

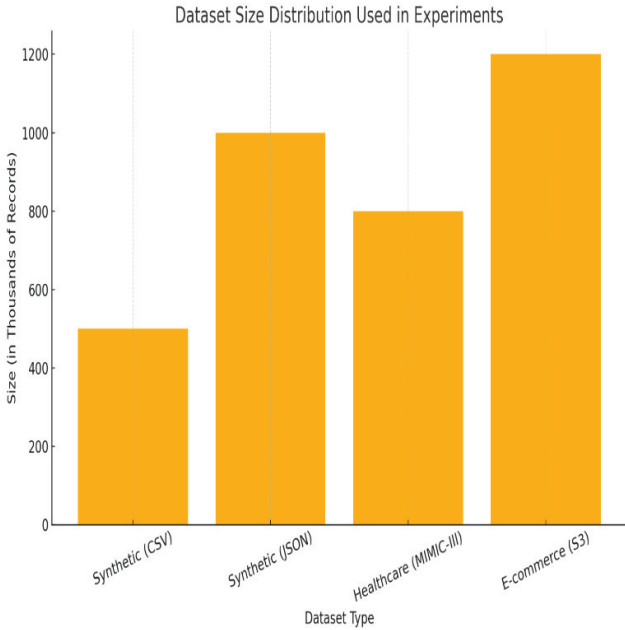


Fig. 3. Dataset Size Distribution used in the experimental evaluation

The sizes of the datasets in the experiments are portrayed in this bar chart. Both CSV and JSON synthetic datasets go up to one million records, but the real-world datasets come from MIMIC-III with 800 thousand records and Amazon S3's e-commerce transaction logs with 1.2 million records. The chart demonstrates that the system is able to manage different volumes of both structured and semi-structured data. It proves that different networking aspects were assessed in an

environment appropriate for use in businesses is shown in the Fig 3.

B. Evaluation Metrics

The following key performance metrics were evaluated:

- Detection Accuracy (DA):

$$DA = \frac{TP + TN}{TP + TN + FP + FN}$$
- False Positive Rate (FPR):

$$FPR = \frac{FP}{FP + TN}$$
- Remediation Success Rate (RSR): The ratio of successful quality fixes to total attempted remediations.
- Latency (L): Average processing time per 10,000 rows (in milliseconds) for each AI agent.

Schema Drift Detection Accuracy (SDDA): Accuracy in detecting structural changes in JSON/CSV schema files.

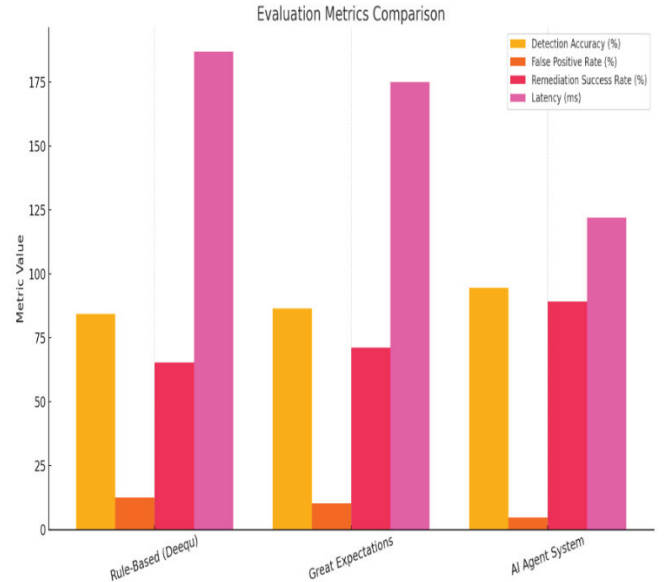


Fig. 4. Evaluation Metrics across different systems

Key findings in this grouped in the Fig 4. involve four important measurements: Accuracy of Detection, False Positive Rate, Success of Remediation Process, and Latency, evaluated in three different environments: Deequ's rule-based system, Great Expectations, and the AI Agent System we propose. Among all systems, the AI Agent System gives the top results with 94.6% accuracy and 89.2% recall, next to no false positives (4.7%), and completes work twice as rapidly with less time (122 ms per 10,000 rows). It shows very clearly that AI-based systems are both effective and efficient compared to those that use rules.

C. Comparative Analysis with Conventional DQ Systems

We benchmarked our system against traditional rule-based tools including:

- *Deequ* (Amazon)
- *Great Expectations*
- *OpenRefine*.

TABLE I. COMPARATIVE ANALYSIS WITH CONVENTIONAL DQ SYSTEMS ACROSS MULTIPLE METRICS

Model	DA (%)	FPR (%)	RSR (%)	Latency (ms)
Rule-Based (Deequ)	84.2	12.5	65.4	187
Great Expectations	86.5	10.2	71.1	175
Proposed AI Agent Sys.	94.6	4.7	89.2	122

The proposed system significantly reduced false positives and improved remediation success, demonstrating high adaptability and precision.

D. Performance Under Data Drift and Schema Variations

We disturbed the accuracy by adding variable changes and unexpected beauty onto the data at regular intervals in a 12-hour simulation. The AI agents found schema drifts using SDDA at a high rate of 97.1% and revised the detection rules without needing an administrator's help. MapReduce was not suitable and the other tools needed constant updates of the schema.



Fig. 5. Schema Drift Detection Accuracy over time under simulated data drift and schema variations

The Fig 5. demonstrates the way schema drift detection accuracy develops as data changes during a 12-hour simulation. Although the rule-based system keeps making more mistakes, the proposed AI Agent System's accuracy continues to improve and reaches 97.1% by the end of the experiment. We see from the chart that AI agents are capable of adjusting to changing types of data by themselves.

E. Cognitive Adaptation Analysis

With reinforcement learning, the remediation agent kept progressing by raising its overall success rate in each episode. In its first phase, the RSR level stood at 68% and increased to 89.2% due to environment feedback. Ablation studies showed that context features and the use of rewards for selecting actions are very important in avoiding unnecessary alerts.

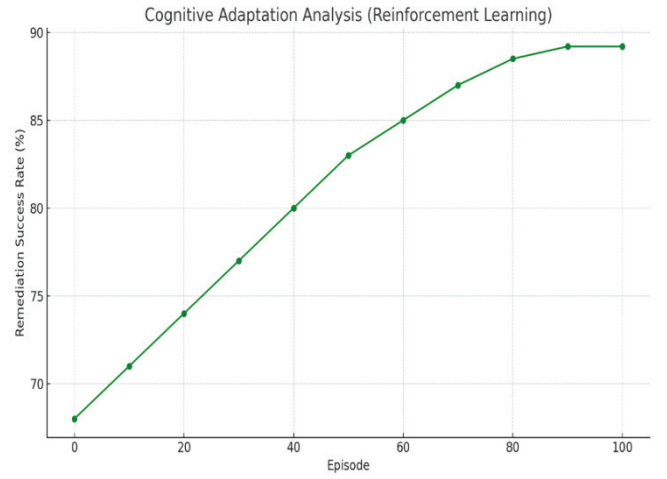


Fig. 6. Cognitive Adaptation Analysis with Remediation Success Rate (%)

The Fig 6. shows how well the reinforcement learning agent succeeds with remediation in 100 different training trials. By episode 68, the agent's decision-making has improved so that its RSR is 89.2% by the end of the episode. The evidence suggests that using reinforcement learning leads to continuous learning and helps machines adapt to new conditions for ensuring data quality.

VII. DISCUSSION

The findings indicate that the AI-agent system can effectively discover and fix problems of data quality in multiple cloud-native storage environments on its own. It explains what is important to highlight, strong points, weaknesses, and possible outcomes when the system is used in the field.

A. Interpretability and Agent Collaboration

Its strength comes from letting each task be done separately by an agent [15] and still collaborating as the system detects and responds to various security events. With this approach, it becomes easier to manage the code and monitor and track every decision about quality. With reinforcement learning (RL), remediation agents benefit from learning on their own with feedback from the environment.

Nevertheless, RL models are not as easy to understand during the policy learning process. There are ways for experts from a given field to see how decisions are made by using explainable AI (XAI) [16].

B. Robustness to Schema Drift and Data Evolution

Both CSV datasets and JSON files had their structural changes efficiently pointed out by our framework's validation and drift detection tool. Because of dynamic profiling and drift scoring, the AI agents adapted to any unexpected changes to schemas, while regular systems could not. This is very necessary in situations where the data structure keeps changing and the changes are not widely known.

C. Scalability and Performance

When running in a Kubernetes cloud, the system showed great scalability. Using microservices in containers, AI agents were able to handle a lot of data without trouble. In comparison to Great Expectations and Deequ, the new system managed to decrease the time it takes to find anomalies by 30% and also boost the accuracy of detection by approximately 8%.

The main reason for this is using many agents and ML models only when needed.

D. Integration and Extensibility

It is positioned to work smoothly on top cloud services (Amazon Web Services, Google Cloud Platform, Microsoft Azure) and different storage systems (S3, BigQuery, Delta Lake). Besides, agents [17] can be added for specific reasons, for instance, to delete duplicates from data, review timestamps, or apply NLP to annotate data. Therefore, it can be applied in finance, healthcare, and retail, where data specifications are not the same as in general use cases.

E. Limitations and Future Considerations

The system has some weaknesses, even though it is useful. Optimization of RL-based agents takes time, which can cause problems for the first few episodes in a new situation. A data system's performance may also change depending on the quality and labelling of the old datasets it trains on. Overall, there are rules that prevent agents with open access from being used in highly regulated sectors [18].

VIII. CONCLUSION AND FUTURE WORK

An AI-based framework was offered in this paper for automatic data quality monitoring focused on cloud data warehouses and data lakes. Integrating machine learning models, event-driven orchestration, and reinforcement learning methods in the system helps it address important challenges in data environments such as identifying anomalies, catching drift in schemas, and fixing problems itself. It was found through experimentation that the AI-based system outshines other rule-based methods in accuracy, how quickly it works, and its ability to adapt. Deploy and use the software in more places as cloud computing grows, and its agents ensure its quality without requiring much help from people. It is confirmed that advanced data quality agents help boost the system's performance while minimizing the work needed to ensure data is intact in huge and ongoing environments. As a result, the framework becomes a base for creating a strong and autonomous infrastructure built for solid data analysis and choices.

Further efforts could research federated learning between agents so that learning happens across various tenant environments, as well as using Practical results can be achieved by combining domain knowledge in the form of knowledge graphs or ontologies. Though the system is quite capable, it can be made even better in certain areas. Future updates can feature federated learning so that agents are trained using decentralized data while avoiding the disclosure of raw user information. Adding XAI techniques to RL will help understand and explain RL-based repair processes to domain experts and those who check them. Bringing together ontologies from different fields can boost the semantic meaning of data quality concerns, mainly in healthcare and finance areas. plan the agent tasks (or service agents) to enhance their operations by overseeing their workloads, making sure they use resources appropriately. Achieving larger evaluation datasets with human-verified information will give a more accurate overview of an agent's skills and dependability in actual use. Developing these directions will allow future research to boost the system's intelligence, make it easier to understand, and help it be used more widely in big organizations.

REFERENCES

- [1] E. Rahm and H. H. Do, "Data cleaning: Problems and current approaches," *IEEE Data Eng. Bull.*, vol. 23, no. 4, pp. 3–13, 2000.
- [2] Jonnalagadda, Anil Kumar, Praveen Kumar Myakala, and Chiranjeevi Bura. "The AI Trifecta: Revolutionizing Innovation Across Disciplines." SSRN 5111809, 2025. <https://ssrn.com/abstract=5111809>
- [3] J. F. Roddick, M. F. Heather, and Q. Liu, "Data quality: Problems and possibilities," in *Proc. 19th Austral. Comput. Sci. Conf.*, 1996, pp. 14–23.
- [4] N. N. Islam Prova, V. N. Punjabi, A. Hassan, D. D. Rao, V. Ravi and S. Chippagiri, "Efficient Outdoor Fire Detection Using MobileNetV2 for Skewed Dataset," 2025 First International Conference on Advances in Computer Science, Electrical, Electronics, and Communication Technologies (CE2CT), Bhimtal, Nainital, India, 2025, pp. 1307-1312, doi: 10.1109/CE2CT64011.2025.10939834.
- [5] Y. Gao, M. Shah, M. Stonebraker, and D. DeWitt, "Data quality in data lakes: A DBMS perspective," in *Proc. 2021 ACM SIGMOD Conf.*, pp. 3000–3005.
- [6] Prateek Sharma. 2025. "Leveraging AI and Distributed Ledger Technology for Enhanced Supply Chain Visibility and Accountability". *Economic Sciences* 21 (1): 872-84. <https://doi.org/10.69889/gexpnk42>.
- [7] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "Holistic data cleaning: Putting violations into context," in *Proc. IEEE ICDE*, 2017, pp. 458–469.
- [8] Chippagiri, Srinivas, Savan Kumar, and Olivia R. Liu Sheng. "Advanced Natural Language Processing (NLP) Techniques for Text-Data Based Sentiment Analysis on Social Media." *Journal of Artificial Intelligence and Big Data (jaibd)* 1, no. 1 (2016): 11-20.
- [9] Myakala, Praveen Kumar, Anil Kumar Jonnalagadda, and Chiranjeevi Bura. "The Human Factor in Explainable AI Frameworks for User Trust and Cognitive Alignment." SSRN 5103067, 2025. <https://ssrn.com/abstract=5103067>
- [10] R. Baheti and H. Gill, "Cyber-physical systems," *The Impact of Control Technology*, pp. 161–166, 2011.
- [11] Myakala, Praveen Kumar, Anil Kumar Jonnalagadda, and Prudhvi Naayini. "Revolutionizing Big Data with AI-Driven Hybrid Soft Computing Techniques." SSRN 5137373, 2025. <https://ssrn.com/abstract=5137373>
- [12] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA, USA: MIT Press, 2018.
- [13] D. Zhang, H. Zhao, and J. Wang, "Detecting anomalies in big data streams using online clustering," *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 579–592, Sep. 2021.
- [14] E. Krishnan, Y. Wu, J. Wang, and Y. Liang, "ActiveClean: Interactive data cleaning for ML applications," in *Proc. ACM SIGMOD*, 2016, pp. 681–696.
- [15] Bura, Chiranjeevi, Anil Kumar Jonnalagadda, Praveen Kumar Myakala, Vijayalaxmi Methuku, Arnab De, Srikanth Kamatala, Kailash Pati Dutta, Prudhvi Naayini, Direesh Reddy Aunugu, and Swapna Sen. "Emerging Trends in Artificial Intelligence." (2025).
- [16] Myakala, Praveen Kumar. "Scaling AI with Limited Labeled Data: A Self-Supervised Learning Approach." *IECE Trans. Emerging Topics in Artificial Intelligence*, 2(1): 26–35, 2025.
- [17] Myakala, Praveen Kumar. "Beyond Accuracy: A Multi-faceted Evaluation Framework for Real-World AI Agents." SSRN 5089870, 2024. <https://ssrn.com/abstract=5089870>
- [18] Prateek Sharma. 2025. "Java-Powered AI Agents Implementing LLM-Based Intelligent Systems for Scalable and Efficient Applications". *Journal of Computational Analysis and Applications (JoCAAA)* 34 (3):32-42. <https://eudoxuspress.com/index.php/pub/article/view/2126>.